KLASIFIKASI CITRA BUAH DAN SAYURAN MENGGUNAKAN METODE KNN

Fadhila Eka Cesary Putri | 2318040

Computer Vision

LATAR BELAKANG

Buah dan sayuran merupakan objek yang memiliki ciri visual yang cukup jelas, terutama dari segi warna. Setiap jenis buah dan sayur memiliki warna dominan yang dapat digunakan sebagai pembeda, seperti apel yang cenderung merah, pisang berwarna kuning, dan timun yang berwarna hijau. Oleh karena itu, fitur warna menjadi salah satu pendekatan yang paling sederhana dan efektif untuk klasifikasi citra.

Pada penelitian ini, digunakan fitur warna dalam ruang RGB untuk merepresentasikan citra buah dan sayuran. Selanjutnya, diterapkan algoritma K-Nearest Neighbor (KNN) untuk mengklasifikasikan objek berdasarkan kemiripan fitur warna yang dimiliki. Metode ini dipilih karena mudah dipahami dan sesuai untuk implementasi sistem klasifikasi sederhana.

RUMUSAN MASALAH

PERTAMA

Bagaimana cara mengekstraksi fitur warna RGB dari citra buah dan sayuran?

KEDUA

Bagaimana cara mengklasifikasikan citra menggunakan metode KNN?

KETIGA

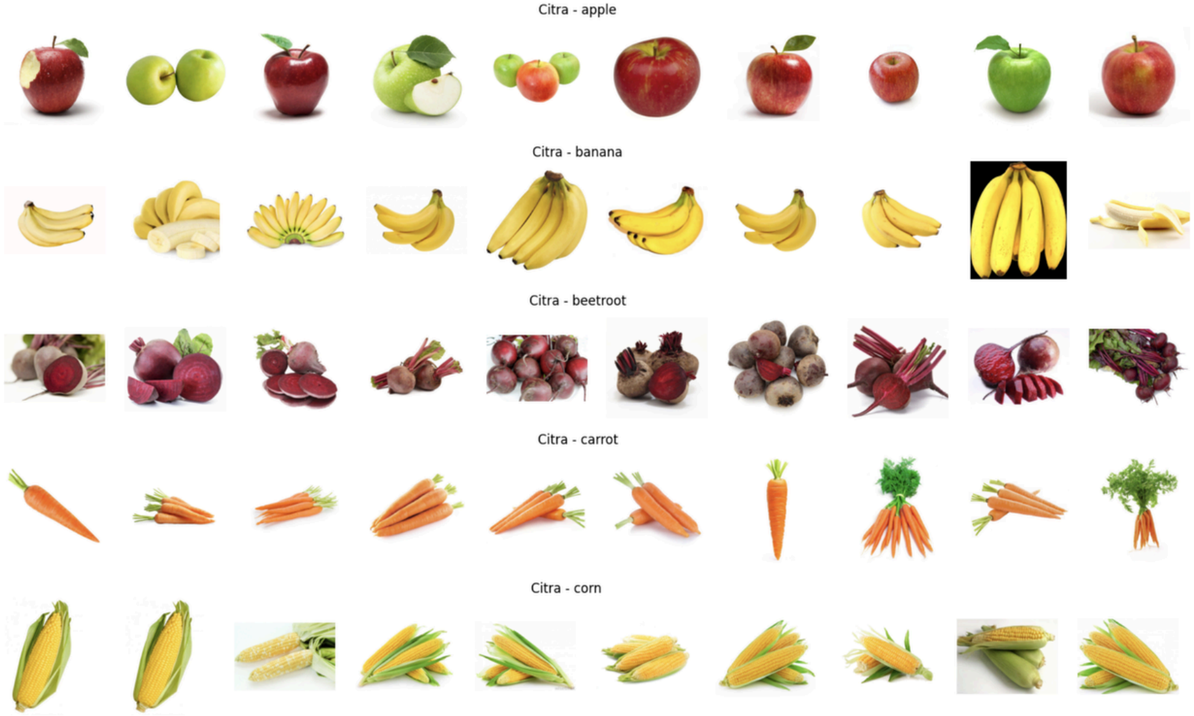
Bagaimana tingkat akurasi yang dihasilkan dari sistem tersebut?

MANFAAT & TUJUAN

Penelitian ini bertujuan untuk mengimplementasikan ekstraksi fitur warna RGB dari citra buah dan sayuran serta mengklasifikasikannya menggunakan metode K-Nearest Neighbor (KNN), sekaligus mengukur tingkat akurasi yang dihasilkan. Adapun manfaat dari penelitian ini adalah memberikan pemahaman dasar mengenai penerapan Machine Learning dalam pengolahan citra digital serta menjadi referensi dalam pengembangan sistem klasifikasi objek berbasis warna secara sederhana.

DATASET

Link Dataset : https://www.google.com/url?q=https%3A%2F%2Fwww.kaggle.com%2Fdatasets%2Fkritikseth%2Ffruit-and-vegetable-image-recognition%3Fselect%3Dtest



ANALISIS CODING

IMPORT LIBRARY

Digunakan untuk mendukung seluruh proses klasifikasi citra buah dan sayuran berbasis Machine Learning. Library tersebut berfungsi untuk membaca dan mengolah gambar (cv2, PIL), mengambil dan mengelola data fitur warna (numpy, pandas), mengatur dataset dari folder (os), menampilkan hasil visualisasi (matplotlib, seaborn), serta membangun dan mengevaluasi model klasifikasi menggunakan algoritma K-Nearest Neighbor (KNN). Dengan adanya import library ini, seluruh proses mulai dari input citra hingga menghasilkan prediksi dapat dilakukan secara lebih mudah, cepat, dan terstruktur.

```
import cv2
import numpy as np
import pandas as pd
import os
import re
import math
import matplotlib.pyplot as plt
import seaborn as sns

from PIL import Image
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
```

MOUNT DRIVE

Digunakan untuk menghubungkan Google Colab dengan Google Drive. Dengan menjalankan drive.mount('/content/drive'), kita dapat mengakses file yang ada di Google Drive secara langsung dari Colab, seperti dataset gambar buah dan sayuran yang digunakan dalam studi kasus

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

ANALISIS CODING

LOAD DATASET

Digunakan untuk mengambil dataset citra buah dan sayuran dari Google Drive, mengekstraknya, lalu menampilkan gambar berdasarkan kategori sebagai tahap awal sebelum proses Machine Learning dilakukan. Bagian zipfile berfungsi untuk membuka file ZIP dan mengekstraknya ke folder tertentu, kemudian os.listdir digunakan untuk membaca struktur dataset (folder apel, pisang, dll). Selanjutnya, kode memfilter file gambar berdasarkan ekstensi yang valid, lalu menggunakan cv2 dan matplotlib untuk membaca dan menampilkan citra dalam bentuk grid per kategori. Proses ini penting untuk memastikan dataset sudah benar sebelum digunakan dalam pelatihan model K-Nearest Neighbor (KNN).

Path Dataset (ZIP Pada Drive)

```
zip_path = '/content/drive/MyDrive/COMVIS 2026/UTS/Fruits and Vegetables.zip'
```

Ekstrak ZIP

```
import zipfile

extract_path = '/content/drive/MyDrive/COMVIS 2026/UTS/Fruits and Vegetables'

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)
```

Path Setelah Ekstrak

```
base_data_dir = os.path.join(extract_path, 'Fruits and Vegetables')
dataset_path = base_data_dir
print(os.listdir(dataset_path))

['apple', 'banana', 'beetroot', 'carrot', 'corn', 'cucumber', 'eggplant', 'ginger', 'grapes', 'kiwi', 'lemon', 'mango', 'onion', 'pepper', 'pomegranate', 'raddish', 'spinach', 'sweetpotato', 'tomato', 'watermelon
```

Menampilkan Citra

```
print("\nMenampilkan citra (per folder)...")

# Define valid image extensions globally
valid_ext = ('.jpg', '.jpeg', '.png', '.gif', '.bmp', '.tiff')

# Now base_data_dir should point to the directory containing 'apple', 'banana', etc.
# Get a list of all subfolders (categories) in the base_data_dir
categories = [f for f in os.listdir(base_data_dir) if os.path.isdir(os.path.join(base_data_dir, f))]

for category_folder in categories:
    category_path = os.path.join(base_data_dir, category_folder)

    if os.path.isdir(category_path):
        # List actual image files within the category folder
        image_files = [f for f in os.listdir(category_path) if f.lower().endswith(valid_ext)]

        # Limit to 100 images per category for display if there are too many
        images_to_display = image_files[:100]

        if images_to_display:
            num_images = len(images_to_display)
            cols = min(10, num_images) # Max 10 columns
            rows = math.ceil(num_images / cols)

            plt.figure(figsize=(cols * 2, rows * 2))
            plt.suptitle(f"Citra - {category_folder}")

            for i, img_name in enumerate(images_to_display):
                img_path = os.path.join(category_path, img_name)
                img = cv2.imread(img_path)

                if img is not None:
                    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

                    plt.subplot(rows, cols, i + 1)
                    plt.imshow(img)
                    plt.axis('off')
                else:
                    print(f"Gagal membaca gambar: {img_path}")
            plt.show()
        else:
            print(f"Tidak ada gambar ditemukan di folder: {category_path}")
```

```
Menampilkan citra (per folder)...
```

Citra - apple

Citra - banana

ANALISIS CODING

EKSTRAKSI FITUR WARNA

Digunakan untuk melakukan preprocessing dan ekstraksi fitur warna RGB dari seluruh citra buah dan sayuran dalam dataset sebagai tahap penting dalam proses Machine Learning. Fungsi preprocessing() digunakan untuk mengubah ukuran gambar agar seragam, sedangkan fungsi extract_rgb_features() mengambil nilai rata-rata warna merah (R), hijau (G), dan biru (B) dari setiap citra sebagai representasi fitur. Selanjutnya, program melakukan perulangan pada setiap folder kategori (misalnya apel, pisang, dll), membaca setiap gambar, melakukan preprocessing, lalu mengekstraksi fitur warna. Hasilnya disimpan dalam bentuk tabel menggunakan pandas (DataFrame) yang berisi nama file dan nilai RGB, yang nantinya akan digunakan sebagai data input untuk proses klasifikasi menggunakan metode K-Nearest Neighbor (KNN).

```python
def extract_rgb_features(image):
    # If the image was converted to RGB earlier or needs to be treated as RGB:
    # Using BGR indexing for cv2.imread output directly.
    R = image[:,:,2]
    G = image[:,:,1]
    B = image[:,:,0]

    mean_r = np.mean(R)
    mean_g = np.mean(G)
    mean_b = np.mean(B)

    return [mean_r, mean_g, mean_b]

def preprocessing(image, size=(100, 100)):
    # Resize the image to a fixed size
    return cv2.resize(image, size)

# PROSES KE SEMUA GAMBAR
data = []
columns = ['nama_file', 'mean_r', 'mean_g', 'mean_b'] # Define columns for the DataFrame

# Get a list of all subfolders (categories) in the base_data_dir
categories = [f for f in os.listdir(base_data_dir) if os.path.isdir(os.path.join(base_data_dir, f))]

for category in categories:
    category_path = os.path.join(base_data_dir, category)
    image_files = [f for f in os.listdir(category_path) if f.lower().endswith(valid_ext)]

    for file_name in image_files:
        path = os.path.join(category_path, file_name)
        image = cv2.imread(path)

        if image is None:
            print(f"Gagal membaca: {path}")
            continue

        # preprocessing
        image_prep = preprocessing(image)

        # ekstraksi fitur
        fitur = extract_rgb_features(image_prep)

        # Append the data with the original filename (e.g., 'apple1.jpg')
        data.append([file_name] + fitur)

# Create DataFrame after processing all images
df = pd.DataFrame(data, columns=columns)
```

ANALISIS CODING

SIMPAN KE DATAFRAME

Digunakan untuk mengelompokkan dan mengurutkan data hasil ekstraksi fitur berdasarkan nama file agar lebih rapi dan terstruktur sebelum digunakan dalam proses Machine Learning. Fungsi extract_category() digunakan untuk mengambil nama kategori (misalnya apel, pisang) dari nama file, sedangkan extract_number() digunakan untuk mengambil angka pada nama file agar bisa diurutkan secara berurutan. Setelah itu, data dalam DataFrame diurutkan berdasarkan kategori dan nomor file, lalu kolom tambahan dihapus agar kembali ke bentuk semula. Proses ini membantu memastikan data tersusun dengan baik sebelum digunakan dalam klasifikasi menggunakan metode K-Nearest Neighbor (KNN).

```python
def extract_category(filename):
    # Use regex to find the alphabetical part before the number
    match = re.match(r'([a-zA-Z]+)', filename)
    if match:
        return match.group(1)
    return 'unknown'

def extract_number(filename):
    # Use regex to find one or more digits in the filename
    match = re.search(r'\\d+', filename)
    if match:
        return int(match.group())
    return np.nan # Return NaN if no number is found

# Apply the function to create a new column for category
df['category'] = df['nama_file'].apply(extract_category)

# Apply the function to create a new column for numerical sorting
df['file_number'] = df['nama_file'].apply(extract_number)

# Sort the DataFrame first by 'category' and then by 'file_number'
df = df.sort_values(by=['category', 'file_number']).reset_index(drop=True)

# Menghapus kolom 'file_number' dan 'category' setelah digunakan untuk sorting
df = df.drop(columns=['file_number', 'category'])

print("Hasil Ekstraksi Fitur RGB :")
display(df.head(10)) # Display more rows to show different categories
```

```
Hasil Ekstraksi Fitur RGB :
```

	nama_file	mean_r	mean_g	mean_b
0	apple1.jpg	214.3184	185.1352	179.7324
1	apple10.png	198.5494	206.5108	108.6227
2	apple2.jpg	211.1428	180.5377	180.5166
3	apple3.jpg	211.2724	219.8044	176.6624
4	apple4.png	233.2770	213.1559	172.0766
5	apple5.jpg	198.9645	133.1978	129.1858
6	apple6.jpg	217.8457	173.7309	165.7894
7	apple7.jpg	235.4548	203.2678	198.0497
8	apple8.jpg	202.2954	220.9097	178.2329
9	apple9.jpg	219.8259	169.8575	160.5997

ANALISIS CODING

SPLIT DATA

Digunakan untuk menyiapkan data sebelum proses klasifikasi dalam Machine Learning. Variabel X berisi fitur warna RGB (mean_r, mean_g, mean_b) sebagai input, sedangkan y berisi label kelas (nama buah atau sayuran) yang diambil dari nama file. Selanjutnya, data dibagi menjadi data training (70%) dan data testing (30%) menggunakan train_test_split, dengan stratify=y agar proporsi tiap kelas tetap seimbang. Proses ini penting agar model K-Nearest Neighbor (KNN) dapat dilatih dengan baik dan diuji secara adil menggunakan data yang belum pernah dilihat sebelumnya.

```python
# Definisikan X sebagai fitur warna (mean_r, mean_g, mean_b)
X = df[['mean_r', 'mean_g', 'mean_b']]

# Definisikan y sebagai label kelas (nama buah/sayuran)
# Fungsi extract_category sudah didefinisikan sebelumnya
def extract_category(filename):
    match = re.match(r'([a-zA-Z]+)', filename)
    if match:
        return match.group(1)
    return 'unknown'
y = df['nama_file'].apply(extract_category)

# Membagi data: 70% training, 30% testing
X_train, X_test, y_train, y_test = train_test_split(
    X,              # fitur
    y,              # label
    test_size=0.3,  # 30% untuk testing
    random_state=42, # agar hasil konsisten
    stratify=y      # menjaga proporsi kelas
)

print(f"Data training: {X_train.shape}")
print(f"Data testing: {X_test.shape}")
print(f"Jumlah kelas training: {len(set(y_train))}")
print(f"Jumlah kelas testing: {len(set(y_test))}")
```

```
Data training: (140, 3)
Data testing: (60, 3)
Jumlah kelas training: 20
Jumlah kelas testing: 20
```

ANALISIS CODING

NORMALISASI DATA

Digunakan untuk melakukan normalisasi data fitur sebelum proses klasifikasi dalam Machine Learning. Dengan menggunakan StandardScaler, data fitur RGB diubah agar memiliki skala yang sama (rata-rata mendekati 0 dan standar deviasi mendekati 1), sehingga tidak ada fitur yang mendominasi perhitungan jarak. Proses fit_transform dilakukan pada data training untuk menghitung parameter skala, sedangkan data testing hanya di-transform menggunakan parameter yang sama agar konsisten. Langkah ini sangat penting karena metode K-Nearest Neighbor (KNN) bekerja berdasarkan perhitungan jarak, sehingga hasil klasifikasi akan lebih akurat jika data sudah dinormalisasi.

```python
# Buat objek scaler
scaler = StandardScaler()

# Fit dan transform data training
X_train_scaled = scaler.fit_transform(X_train)

# Transform data testing (pakai scaler yang sama)
X_test_scaled = scaler.transform(X_test)

print("Sebelum normalisasi:")
print(f"Rata-rata fitur (sampel pertama): {X_train.iloc[0].values}")
print(f"Standar deviasi: {np.std(X_train.values, axis=0)}")

print("\nSetelah normalisasi:")
print(f"Rata-rata fitur (sampel pertama): {X_train_scaled[0][:5]}")
print(f"Standar deviasi: {np.std(X_train_scaled[:, :5], axis=0)}")
```

```
Sebelum normalisasi:
Rata-rata fitur (sampel pertama): [157.9413 151.1361 153.7255]
Standar deviasi: [35.06401238 38.24582089 42.26305009]

Setelah normalisasi:
Rata-rata fitur (sampel pertama): [-1.5502196  -1.12621446 -0.32369905]
Standar deviasi: [1. 1. 1.]
```

ANALISIS CODING

TRAINING KNN DENGAN SATU NILAI K

Digunakan untuk membuat dan melatih model klasifikasi menggunakan algoritma K-Nearest Neighbor (KNN) dalam proses Machine Learning. Model dibuat dengan parameter k=5, yang berarti sistem akan mempertimbangkan 5 tetangga terdekat dalam menentukan kelas suatu data. Selanjutnya, model dilatih menggunakan data training yang telah dinormalisasi (X_train_scaled dan y_train) agar dapat mengenali pola dari fitur warna RGB. Setelah proses ini selesai, model siap digunakan untuk melakukan prediksi pada data baru.

```python
# Buat model KNN dengan k=5
knn = KNeighborsClassifier(n_neighbors=5)

# Latih model dengan data training
knn.fit(X_train_scaled, y_train)

print("Model KNN berhasil dilatih!")
print(f"Jumlah tetangga (k) = 5")
```

```
Model KNN berhasil dilatih!
Jumlah tetangga (k) = 5
```

ANALISIS CODING

PREDIKSI DATA TESTING

Digunakan untuk melakukan prediksi dan melihat hasil klasifikasi pada data testing menggunakan model K-Nearest Neighbor (KNN) dalam proses Machine Learning. Pertama, model memprediksi label dari data uji (X_test_scaled) dan hasilnya disimpan dalam y_pred. Kemudian ditampilkan 10 hasil prediksi pertama untuk membandingkan antara label asli (True) dan hasil prediksi (Predicted). Selain itu, kode juga menampilkan probabilitas prediksi untuk beberapa data, yang menunjukkan tingkat keyakinan model terhadap masing-masing kelas. Hal ini berguna untuk memahami seberapa yakin model dalam menentukan hasil klasifikasi.

```python
# Prediksi semua data testing
y_pred = knn.predict(X_test_scaled)

# Tampilkan 10 hasil prediksi pertama
print("10 Hasil Prediksi Pertama:")
for i in range(10):
    print(f"Data {i+1}: True={y_test.iloc[i]}, Predicted={y_pred[i]}")

# Tampilkan probabilitas prediksi untuk 3 data pertama
y_proba = knn.predict_proba(X_test_scaled[:3])
print("\nProbabilitas prediksi (3 data pertama):")
print(y_proba)
```

```
10 Hasil Prediksi Pertama:
Data 1: True=raddish, Predicted=raddish
Data 2: True=onion, Predicted=beetroot
Data 3: True=watermelon, Predicted=apple
Data 4: True=grapes, Predicted=kiwi
Data 5: True=carrot, Predicted=carrot
Data 6: True=corn, Predicted=corn
Data 7: True=ginger, Predicted=carrot
Data 8: True=beetroot, Predicted=beetroot
Data 9: True=cucumber, Predicted=cucumber
Data 10: True=apple, Predicted=spinach

Probabilitas prediksi (3 data pertama):
[[0.  0.  0.  0.  0.  0.2 0.2 0.2 0.  0.  0.  0.  0.  0.  0.  0.4 0.  0.
  0.  0. ]
 [0.  0.  0.4 0.  0.  0.  0.4 0.  0.  0.  0.  0.  0.  0.  0.2 0.  0.  0.
  0.  0. ]
 [0.2 0.  0.  0.  0.  0.  0.  0.2 0.  0.  0.  0.  0.2 0.  0.  0.  0.  0.2
  0.2 0. ]]
```

ANALISIS CODING

EVALUASI AKURASI

Digunakan untuk mengevaluasi kinerja model klasifikasi dalam proses Machine Learning. Fungsi accuracy_score digunakan untuk menghitung tingkat akurasi model K-Nearest Neighbor (KNN), yaitu perbandingan antara jumlah prediksi yang benar dengan total data uji. Selain itu, kode juga menghitung jumlah prediksi yang benar dan salah untuk memberikan gambaran lebih jelas mengenai performa model. Hasil ini membantu mengetahui seberapa baik model dalam mengenali citra buah dan sayuran berdasarkan fitur warna RGB yang telah diekstraksi.

```python
# Hitung akurasi
akurasi = accuracy_score(y_test, y_pred)

print(f"Akurasi model KNN (k=5): {akurasi:.4f}")
print(f"Persentase: {akurasi * 100:.2f}%")

# Hitung jumlah benar dan salah
jumlah_benar = sum(y_pred == y_test)
jumlah_salah = sum(y_pred != y_test)
total_data = len(y_test)

print(f"\nDari {total_data} data testing:")
print(f"✓ Prediksi benar: {jumlah_benar} data")
print(f"✗ Prediksi salah: {jumlah_salah} data")
```

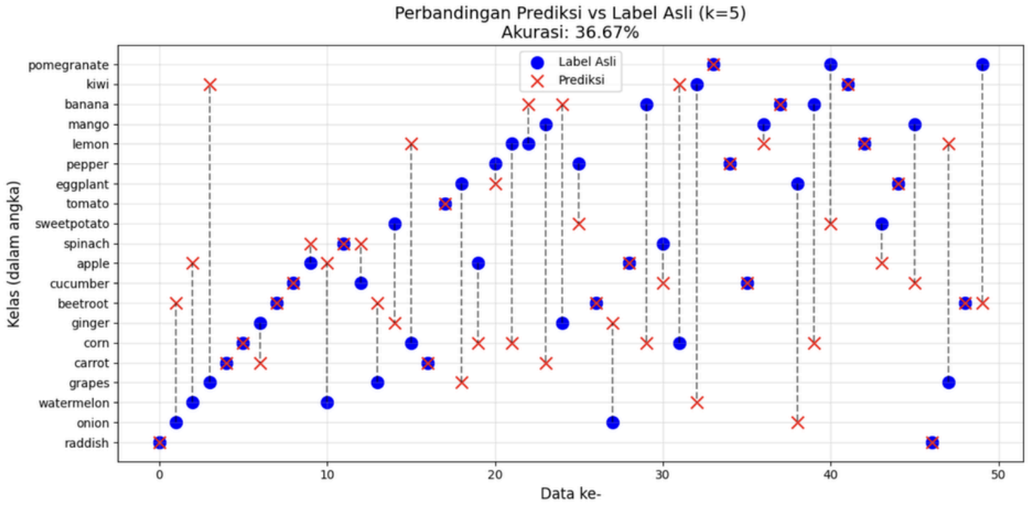
```
Akurasi model KNN (k=5): 0.3667
Persentase: 36.67%

Dari 60 data testing:
✓ Prediksi benar: 22 data
✗ Prediksi salah: 38 data
```

VISUALISASI DATA

PERBANDINGAN PREDIKSI VS LABEL ASLI

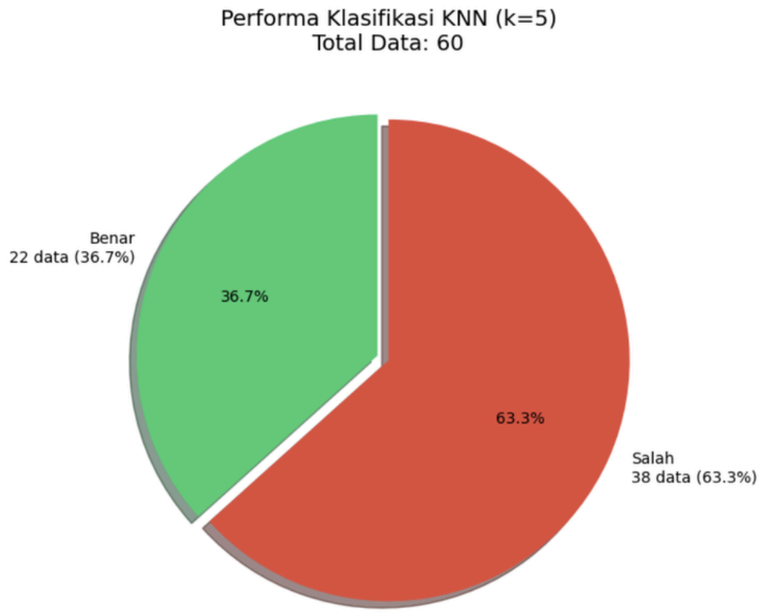
Diagram disamping menunjukkan perbandingan antara label asli dan hasil prediksi menggunakan algoritma K-Nearest Neighbor, di mana titik biru adalah label asli dan tanda silang merah adalah hasil prediksi. Jika keduanya sejajar berarti prediksi benar, sedangkan jika berbeda berarti terjadi kesalahan. Dari grafik terlihat banyak ketidaksesuaian, sehingga menghasilkan akurasi sekitar 36.67%, yang menunjukkan bahwa model masih kurang optimal dalam mengklasifikasikan citra buah dan sayuran berdasarkan fitur warna RGB.



VISUALISASI DATA

DISTRIBUSI PREDIKSI BENAR VS SALAH

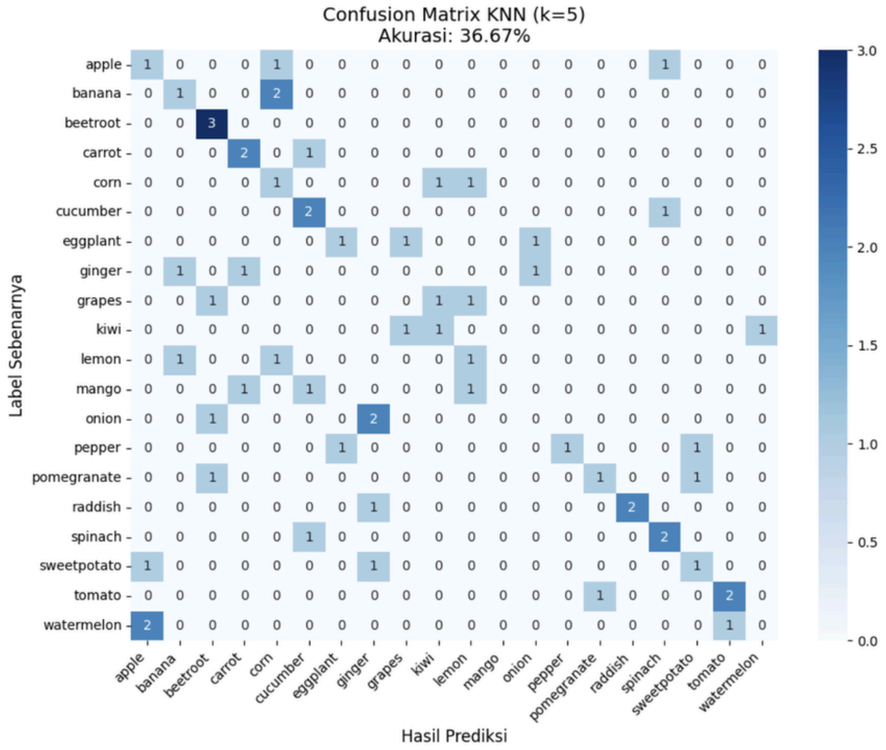
Diagram disamping menunjukkan perbandingan jumlah prediksi benar dan salah dari model K-Nearest Neighbor (KNN), di mana dari total 60 data uji hanya 22 data (36.7%) yang diprediksi dengan benar dan 38 data (63.3%) salah. Hal ini menunjukkan bahwa performa model masih rendah, sehingga metode yang digunakan atau fitur warna RGB yang dipakai belum mampu mengklasifikasikan citra buah dan sayuran dengan baik.



VISUALISASI DATA

CONFUSION MATRIX

Diagram disamping merupakan confusion matrix yang menunjukkan performa model K-Nearest Neighbor dalam mengklasifikasikan citra buah dan sayuran. Nilai pada diagonal menunjukkan prediksi yang benar, sedangkan nilai di luar diagonal menunjukkan kesalahan klasifikasi antar kelas. Terlihat bahwa masih banyak data yang tidak berada di diagonal, sehingga menghasilkan akurasi sekitar 36.67%, yang menandakan model masih kurang optimal dalam membedakan berbagai jenis buah dan sayuran berdasarkan fitur warna RGB.



KESIMPULAN

Penelitian ini menunjukkan bahwa ekstraksi fitur warna RGB dapat digunakan untuk merepresentasikan citra buah dan sayuran secara sederhana dalam proses Machine Learning. Dengan menggunakan algoritma K-Nearest Neighbor (KNN), sistem mampu melakukan klasifikasi berdasarkan kemiripan warna, namun hasil yang diperoleh masih memiliki akurasi yang rendah. Hal ini disebabkan oleh keterbatasan fitur RGB yang sensitif terhadap pencahayaan serta kemiripan warna antar objek. Oleh karena itu, diperlukan peningkatan seperti penambahan dataset, penggunaan fitur lain, atau metode yang lebih kompleks agar hasil klasifikasi menjadi lebih optimal.

TERIMA KASIH